CSC3100: Addressing & Array & Sorting

Tianshu Yu

Fall 2024

Overview

- Basics in Computer Memory
  - Partially go beyond the current scope.
- Arrays
- Implementation
- Examples

Structure of memory

- The fundamental unit of memory is called a **bit**, either 0 or 1.

- In most modern architectures, the smallest unit on which the hardware operates is a sequence of eight consecutive bits called **byte**.

  a binary (executable) file

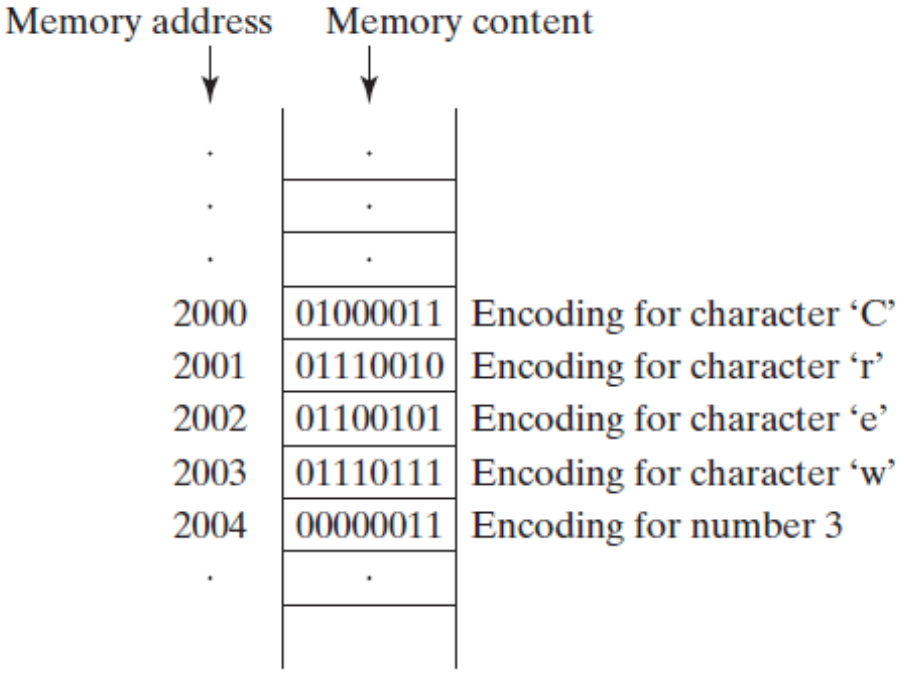
  0 1 2 3

  01011001100001001001111010000011...

- Numbers and instructions are stored in still larger units. The most common integer size on a particular hardware is called a **word**. Because machines have different architectures, the number of bytes and the order of bytes in a word may vary from machine to machine.

Memory and addressing

- A computer's memory consists of an ordered sequence of bytes for storing data
- Every location in the memory has a unique address
- The key difference between high and low level programming languages is whether programmer has to deal with memory addressing directly



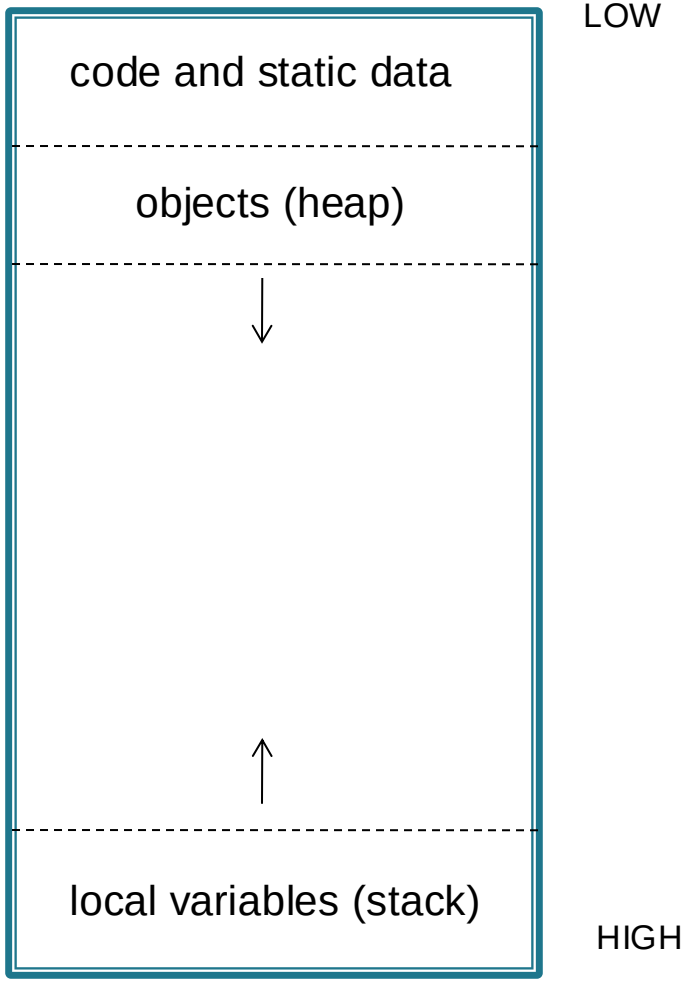
Numbers, bases, and conversion

- $(21)_{10} = (10101)_2$
- $(0.65625)_{10} = (0.10101)_2$

- Octal (0,1,2,3,4,5,6,7)
  $(10101)_2 = (010101)_2 = (25)_8$
  $(0.10101)_2 = (0.101010)_2 = (0.52)_8$

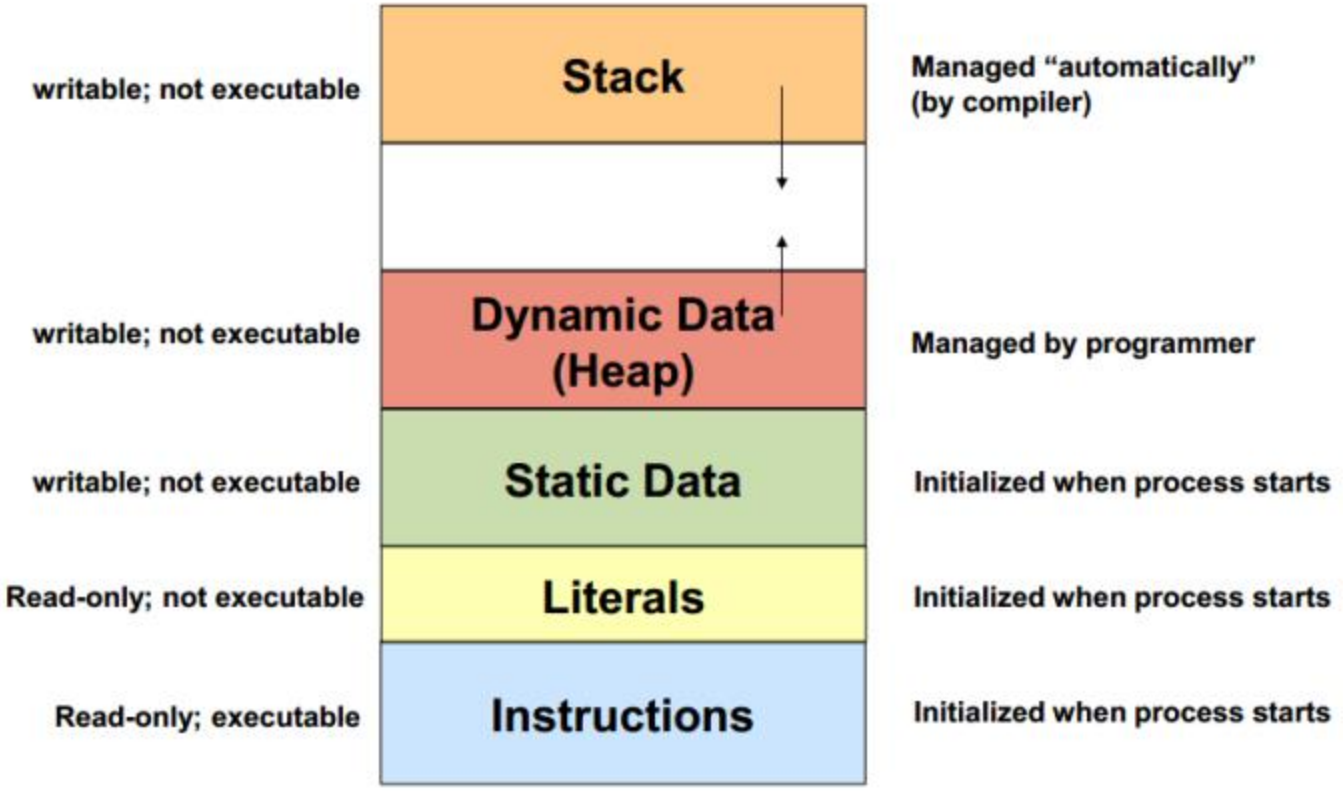
- Hexadecimal (0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F)
  $(10101)_2 = (00010101)_2 = (15)_{16}$
  $(0.10101)_2 = (0.10101000)_2 = (0.A8)_{16}$

- Useful numbers
  $(10000000000)_2 = (1024)_{10}$ (about 1K)



Memory allocation

If you prefer the other direction



Stack vs heap

Parameter	STACK	HEAP
Basic	Memory is allocated in a contiguous block.	Memory is allocated in any random order.
Allocation and De-allocation	Automatic by compiler instructions.	Manual by the programmer.
Cost	Less	More
Implementation	Easy	Hard
Access time	Faster	Slower
Main Issue	Shortage of memory	Memory fragmentation
Locality of reference	Excellent	Adequate
Safety	Thread safe, data stored can only be accessed by owner	Not Thread safe, data stored visible to all threads
Flexibility	Fixed-size	Resizing is possible
Data type structure	Linear	Hierarchical

Will discuss with more details in a later topic.

Memory allocation to variables

- One region of memory is reserved for **static data**
  - never created or destroyed as program runs, such as named constants.
- When a new object is created, Java allocates space from **heap**.
- When a method is called, Java allocates a new block of memory called a stack frame to hold its local variables.
- When a method returns, its stack frame is erased. Stack frames come from **stack**.

Object references

- Java identifies an object by its address in memory. That address is called a reference.

- Eg., when Java evaluates the declaration

  ```
  Rational a = new Rational(1, 2);
  ```

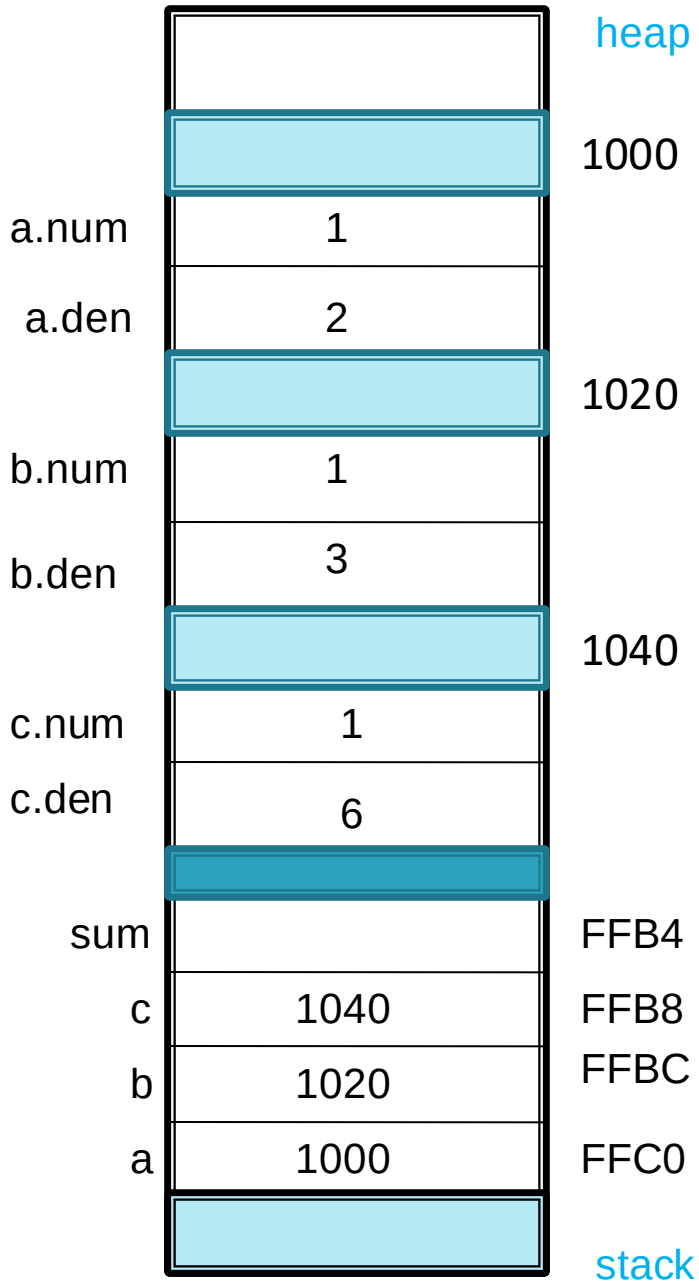
  it allocates heap space for the new Rational object. For this example, imagine that the object is allocated at address 1000.

- The local variable `a` is allocated in the current stack frame and is assigned the value (address), which identifies the object

```java
public class Rational
{
    /** Creates a new Rational initialized to zero */
    public Rational() {
        this(0);
    }
    /**
     * Creates a new Rational from the integer argument.
     * @param n The initial value
     */
    public Rational(int n) {
        this(n, 1);
    }
    /**
     * Creates a new Rational with the value x / y.
     * @param x The numerator of the rational number
     * @param y The denominator of the rational number
     */
    public Rational(int x, int y) {
        int g = gcd(Math.abs(x), Math.abs(y));
        num = x / g;
        den = Math.abs(y) / g;
        if (y < 0) num = -num;
    }
    /**
     * Adds the rational number r to this one and returns the sum.
     * @param r The rational number to be added
     * @return The sum of the current number and r
     */
    public Rational add(Rational r) {
        return new Rational(this.num * r.den + r.num * this.den,
                            this.den * r.den);
    }
    /**
     * Subtracts the rational number r from this one and returns
     * the difference.
     * @param r The rational number to be subtracted
     * @return The result of subtracting r from the current number
     */
    public Rational subtract(Rational r) {
        return new Rational(this.num * r.den - r.num * this.den,
                            this.den * r.den);
    }
    /**
     * Multiplies this number by the rational number r.
     * @param r The rational number used as a multiplier
     * @return The result of multiplying the current number by r
     */
    public Rational multiply(Rational r) {
        return new Rational(this.num * r.num, this.den * r.den);
    }
    /**
     * Divides this number by the nonzero rational number r.
     * @param r The nonzero rational number used as a divisor
     * @return The result of dividing the current number by r
     */
    public Rational divide(Rational r) {
        return new Rational(this.num * r.den, this.den * r.num);
    }
    /**
     * Creates a string representation of this rational number.
     * @return The string representation of this rational number
     */
    public String toString() {
        if (den == 1) {
            return "" + num;
        } else {
            return num + "/" + den;
        }
    }
    /**
     * Calculates the greatest common divisor using Euclid's algorithm.
     * @param First integer
     * @param Second integer
     * @return The greatest common divisor of x and y
     */
    private int gcd(int x, int y) {
        int r = x % y;
        while (r != 0) {
            x = y;
            y = r;
            r = x % y;
        }
        return y;
    }
    /** Private instance variables */
    private int num;      /* The numerator of this Rational */
    private int den;      /* The denominator of this Rational */
} /* class Rational */
```

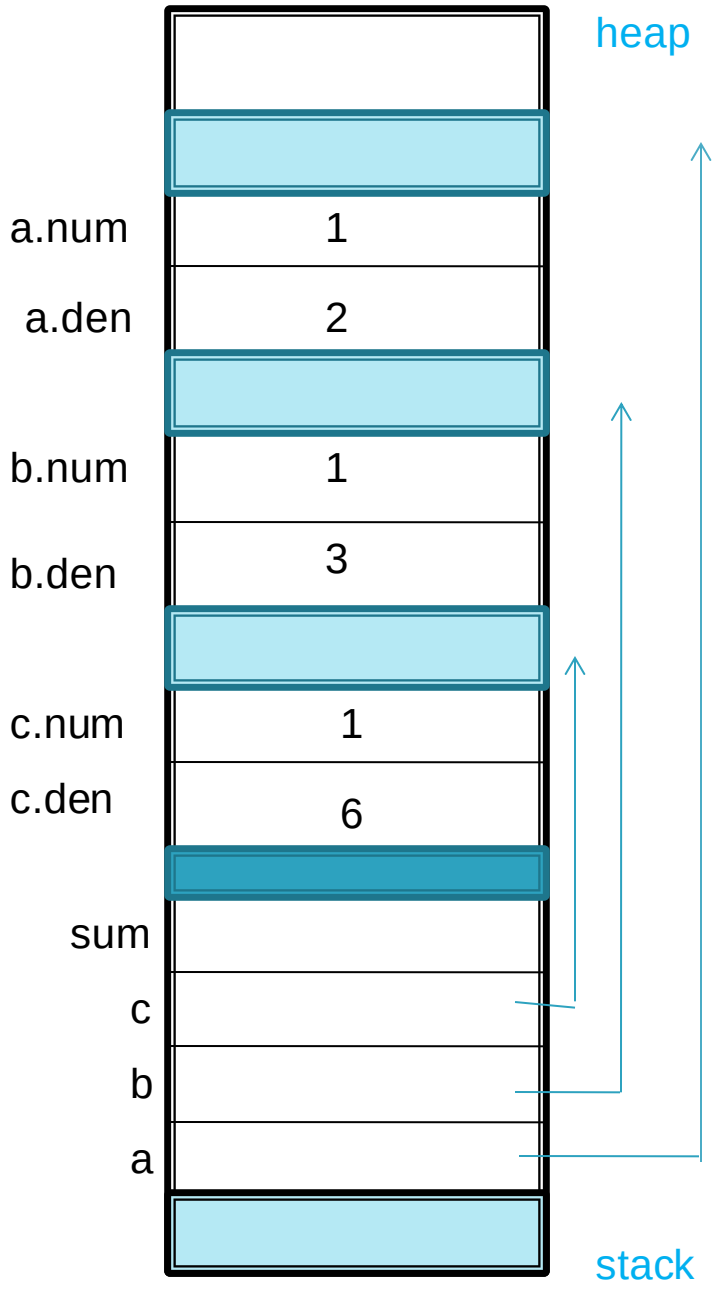
Address Model

```
public void run() {
      Rational  a = new Rational(1, 2);
      Rational  b = new Rational(1, 3);
      Rational  c = new Rational(1, 6);
      Rational  sum = (a.add(b)).add(c);
}
```



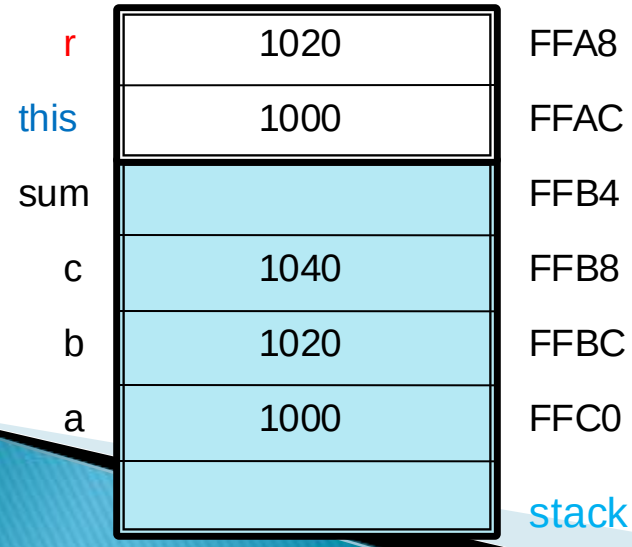
Pointer Model

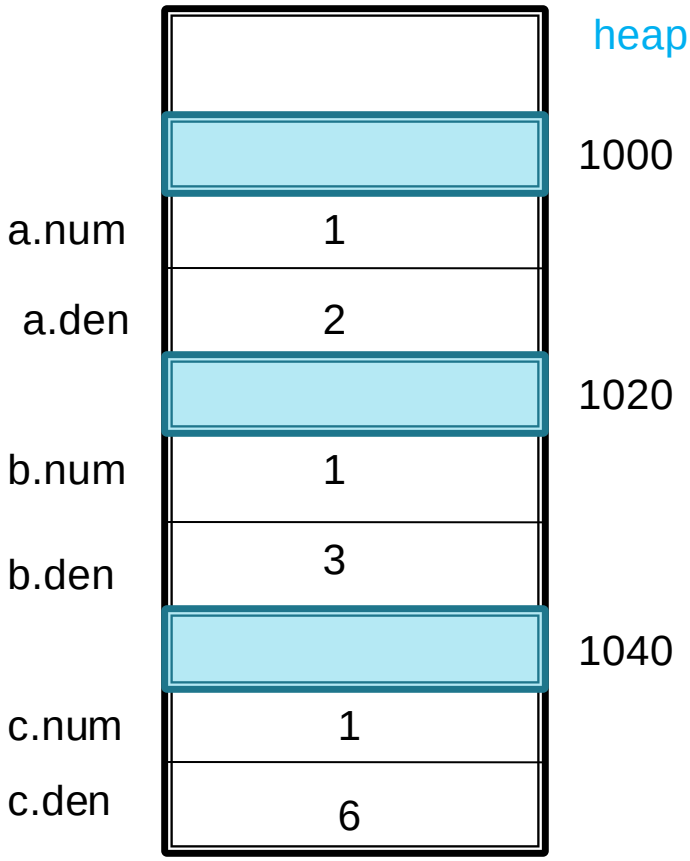
```
public void run() {
      Rational  a = new Rational(1, 2);
      Rational  b = new Rational(1, 3);
      Rational  c = new Rational(1, 6);
      Rational  sum = (a.add(b)).add(c);
}
```



```
public void run() {
        Rational  a = new Rational(1, 2);
        Rational  b = new Rational(1, 3);
        Rational  c = new Rational(1, 6);
        Rational  sum = (a.add(b)).add(c);
}

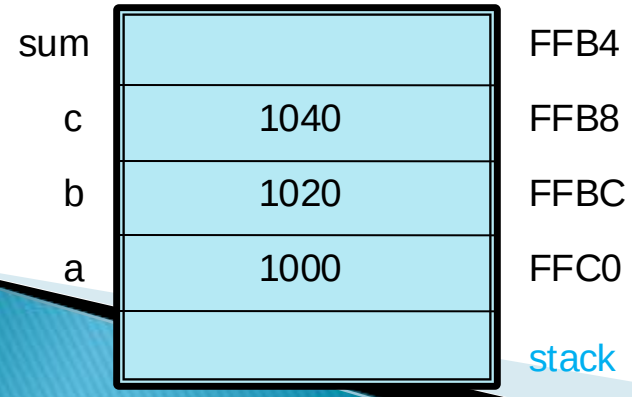
public  Rational  add(Rational  r) {
      return new Rational(this.num*r.den
                              + r.num*this.den,
                             this.den*r.den);
}
```

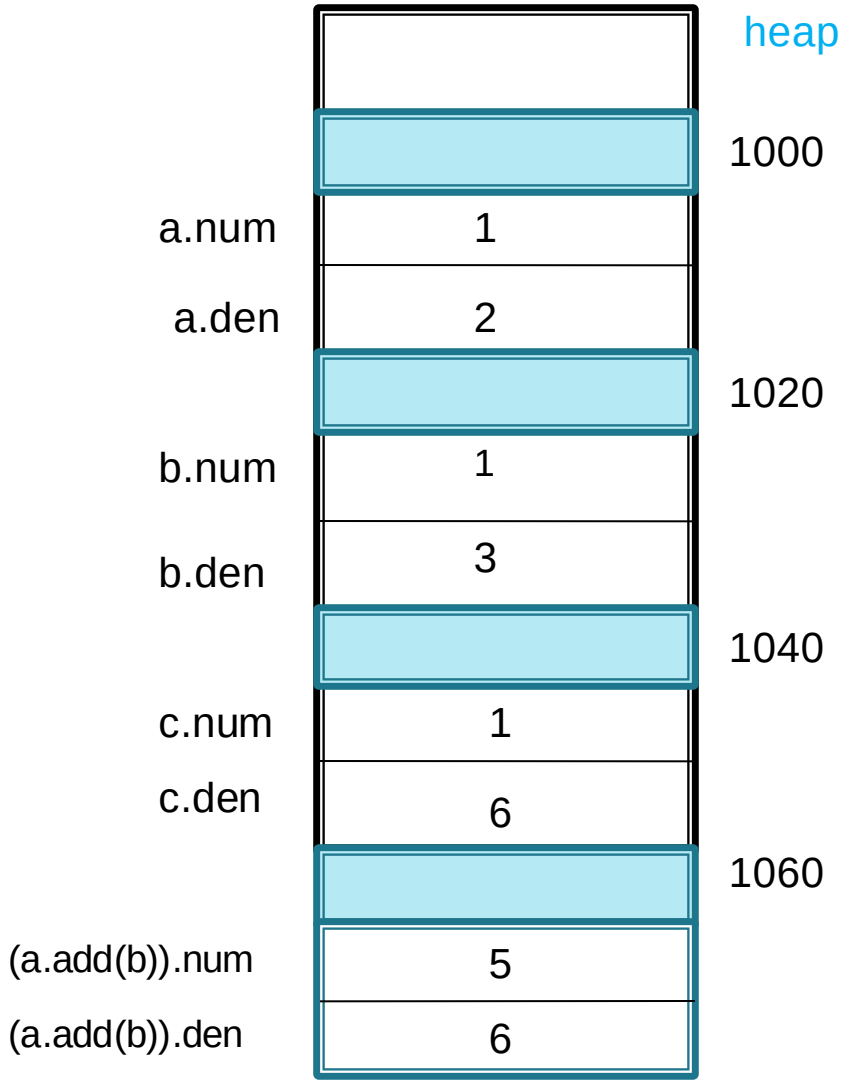




```
public void run() {
        Rational  a = new Rational(1, 2);
        Rational  b = new Rational(1, 3);
        Rational  c = new Rational(1, 6);
        Rational  sum = (a.add(b)).add(c);
}

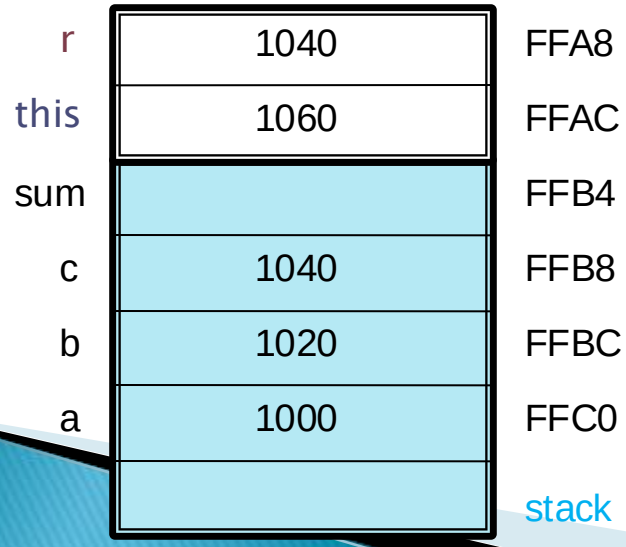
public  Rational  add(Rational  r) {
     return new Rational(this.num*r.den
                              + r.num*this.den,
                             this.den*r.den);
}
```

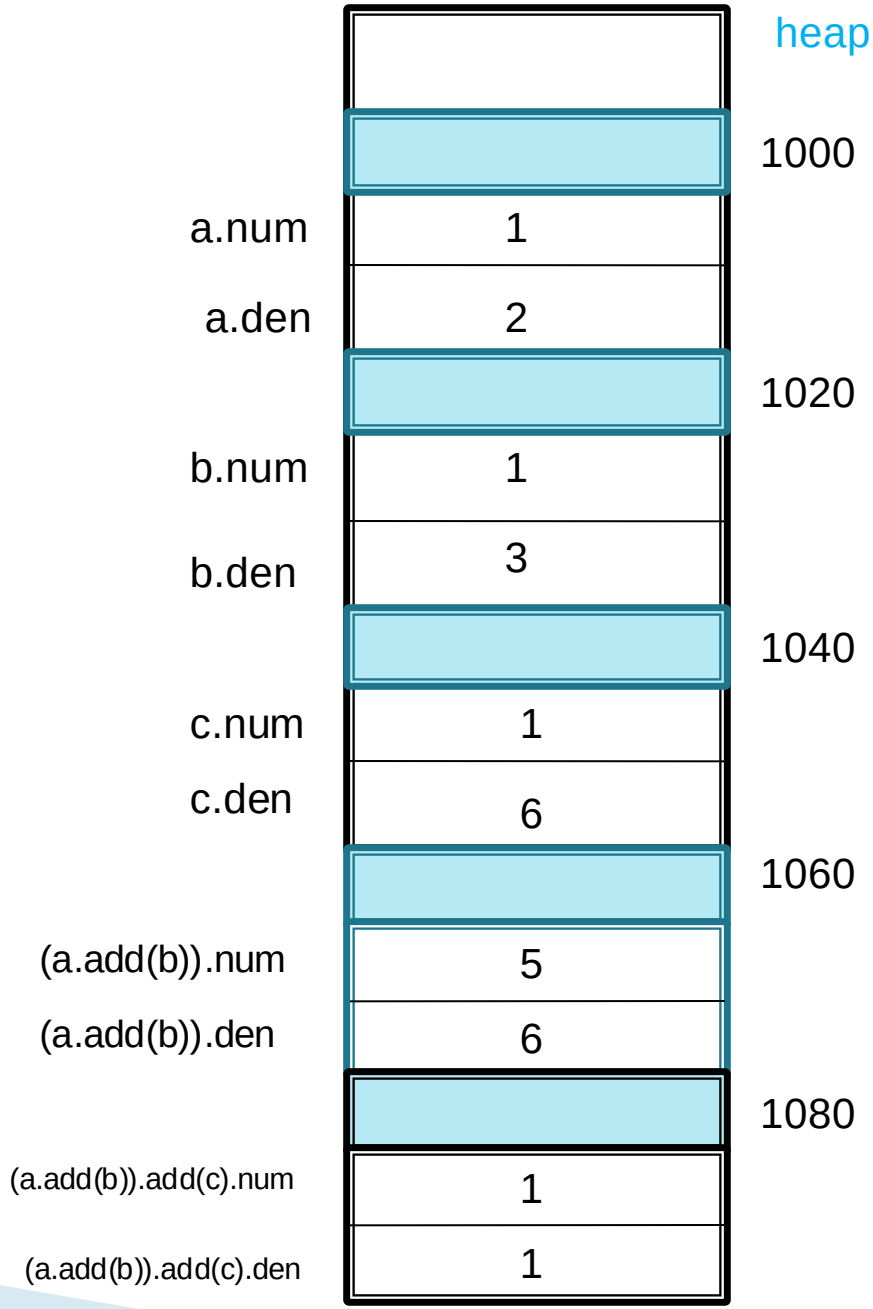




```
public void run() {
      Rational  a = new Rational(1, 2);
      Rational  b = new Rational(1, 3);
      Rational  c = new Rational(1, 6);
      Rational  sum = (a.add(b)).add(c);
}

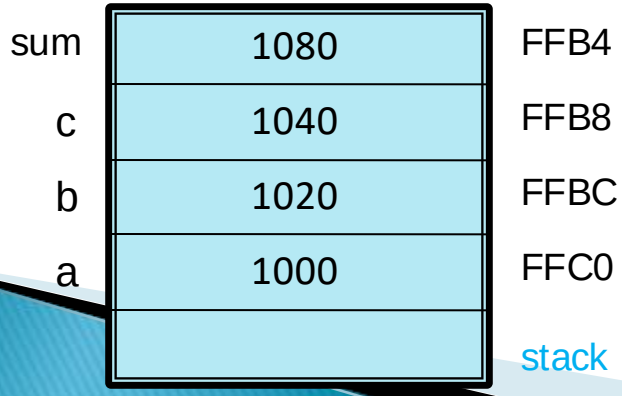
public  Rational  add(Rational  r) {
    return new Rational(this.num*r.den
                        + r.num*this.den,
                       this.den*r.den);
}
```

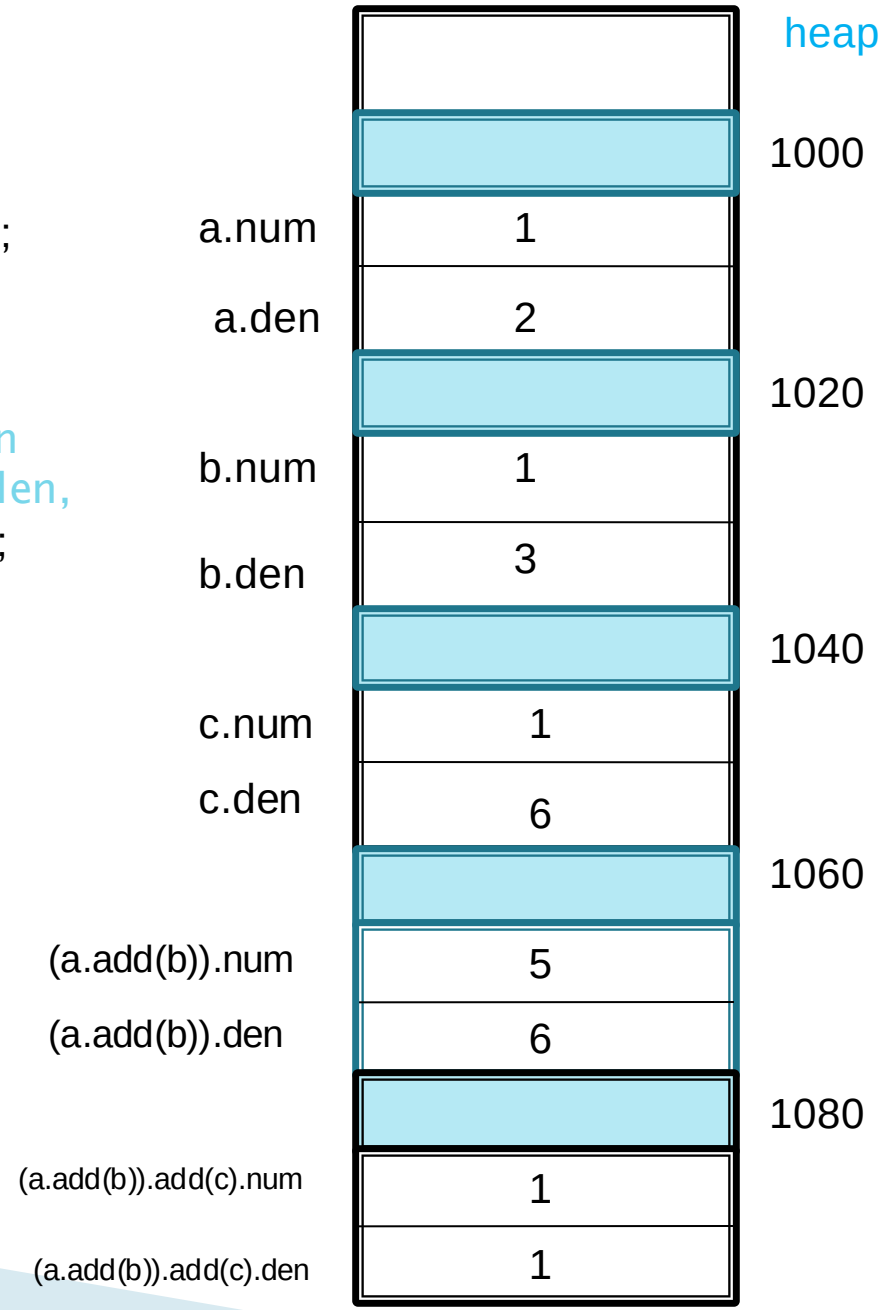




```
public void run() {
      Rational  a = new Rational(1, 2);
      Rational  b = new Rational(1, 3);
      Rational  c = new Rational(1, 6);
      Rational  sum = (a.add(b)).add(c);
}

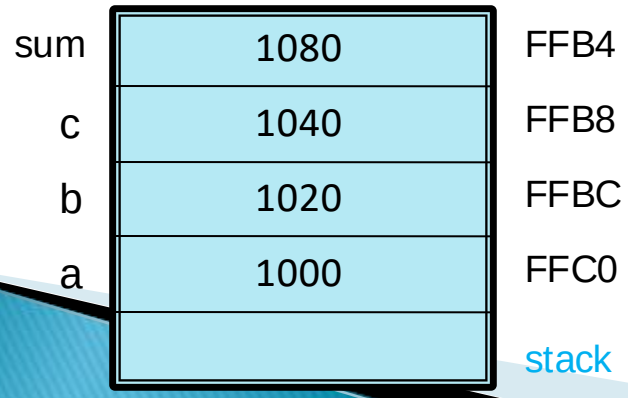
public  Rational  add(Rational  r) {
    return new Rational(this.num*r.den
                        + r.num*this.den,
                       this.den*r.den);
}
```

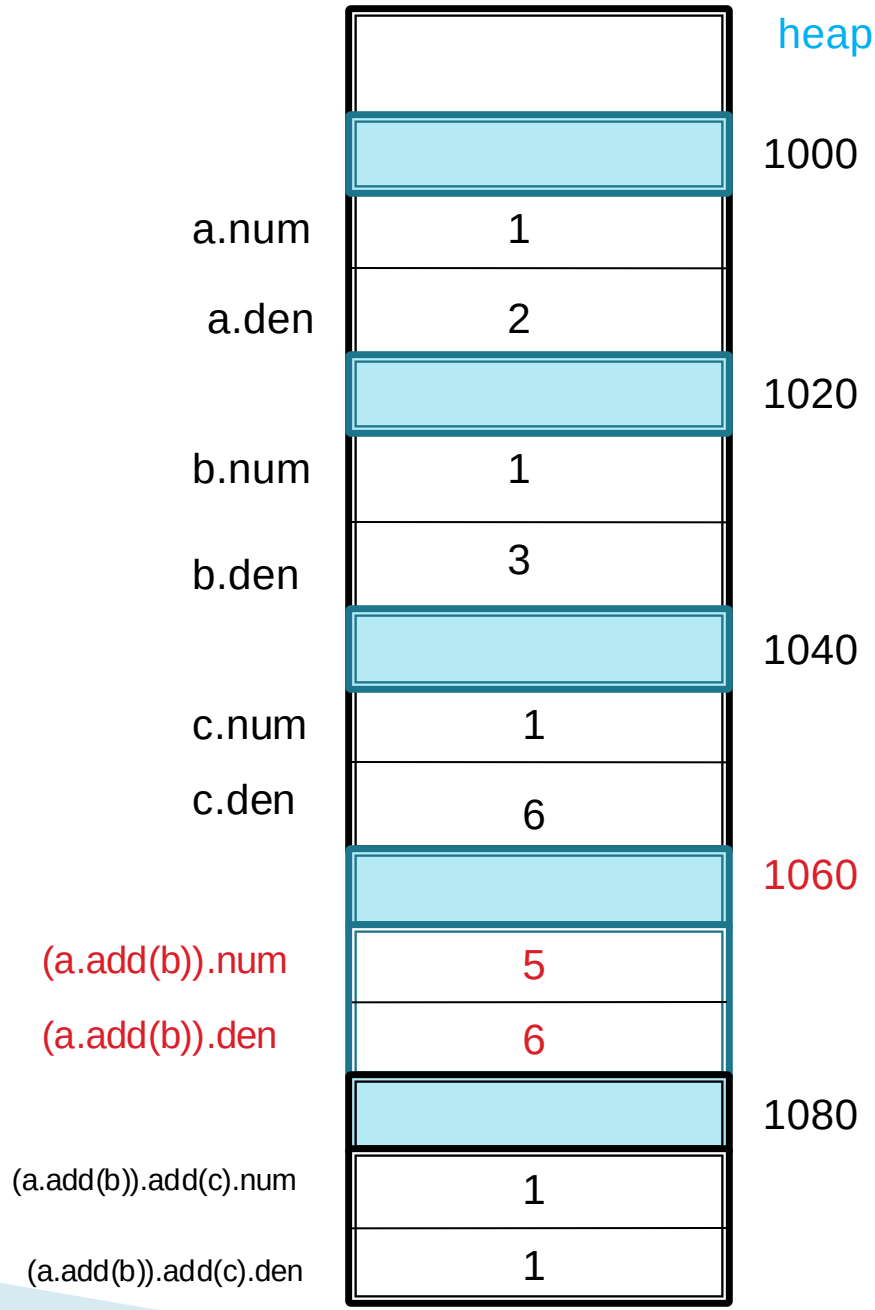




```
public void run() {
        Rational  a = new Rational(1, 2);
        Rational  b = new Rational(1, 3);
        Rational  c = new Rational(1, 6);
        Rational  sum = (a.add(b)).add(c);
}

public  Rational  add(Rational  r) {
     return new Rational(this.num*r.den
                          + r.num*this.den,
                         this.den*r.den);
}
```





Garbage collection

- In the example, the object `a.add(b)` was created in the intermediate step but not referenced by the final stack. It is now **garbage**.

- When memory is running short, Java does garbage collection
  - Mark the objects referenced by variables on stack or in static storage.
  - Sweep all objects in the heap, reclaim unmarked objects (garbage).

- This process is called **garbage collection**.

Primitive type versus objects

- Primitive type

```
public void run() {
       int  x = 17;
       increment(x);
       System.out.println("x = " + x);
}

private void increment(int n) {
     n++;
     System.out.println("n = " + n);
}
```

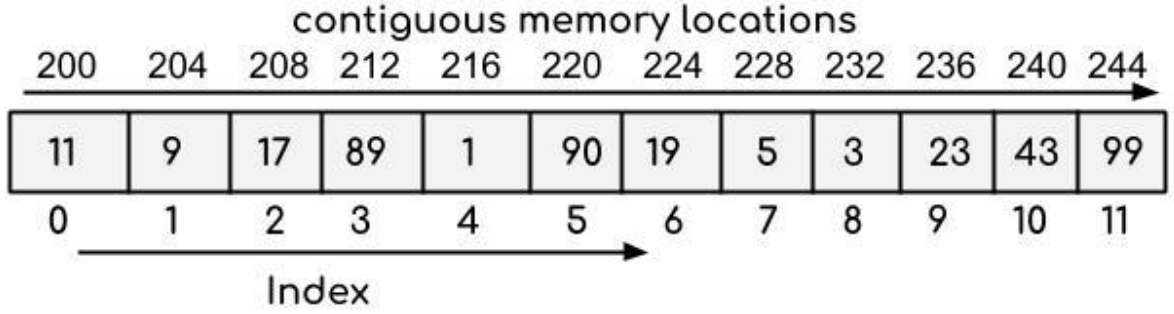
Output
n = 18
n = 17

When you pass an argument of a primitive type to a method, Java copies the value of the argument into the parameter variable. As a result, changes to the parameter variable have no effect on the argument.

outline

- Basics in Computer Memory
- Arrays
- Implementation
- Examples

Array



- Array: An order
  - Ordered and fixed length
  - Homogeneous: Each value in the array is of the same type
- The individual values in an array are called **elements**.
- The number of elements is called the **length** of the array
- Each element is identified by its position in the array, which is called **index**.
  - In Java, the index numbers begin with 0.

Illustration from wikipedia

- Array: a data structure consisting of a collection of elements (values or variables)
  - Each element is identified by at least one array index or key.
  - The memory position of each element can be computed from its index tuple.
  - The simplest type of data structure is a linear array, also called one-dimensional array.

- Example: an array of 10 32-bit (4-byte) integer variables, with indices 0 through 9,
  - May be stored as 10 words at memory addresses 2000, 2004, 2008, ..., 2036, (in hexadecimal: 0x7D0, 0x7D4, 0x7D8, ..., 0x7F4)
  - The element with index i has the address 2000 + (i × 4).

Array declaration

- An array is characterized by
  - Element type
  - Length

```
type[] identifier = new type[length];
```

- Default values in initialization
  - numerics 0
  - boolean false
  - objects null

An array of objects



Elements of an array can be objects of any Java class.



Example: An array of 5 instances of the student class

```
Student [ ]
topStudents = new Student[5];
```

```java
/**
 * The student class is basic class.
 */
public class Student
{
    /**
     * @param name The student's name
     * @param id student's id
     */
    public Student(String name, int id) {
        studentName = name;
        studentId = id;
    }
    /**
     * Gets name of student
     * @return the name of student
     */
    public String getName() {
        return studentName;
    }
    /**
     * Gets id of student
     * @return the id of student
     */
    public int getId() {
        return studentId;
    }
    /**
     * sets the number of credits enarned.
     * @param credits The new number of credits earned
     */
    public void setCredits(double credits) {
        creditsEarned = credits;
    }
    /**
     * Gets the number of credits earned.
     * @return The number of credits this student has earned
     */
    public double getCredits() {
        return creditsEarned;
    }
    /**
     * Sets whether the student is paid up.
     * @param flag The value true or false indicating paid-up status
     */
    public void setPaidUp(boolean flag) {
        paidUp = flag;
    }
    /**
     * Returns whether the sutdent is paid up.
     * @return Whether the student is paid up.
     */
    public boolean isPaidUp() {
        return paidUp;
    }
    /**
     * Creates a string identifying this student.
     * @return The string used to display this student.
     */
    public String toString() {
        return studentName + " (#" + studentId + ")";
    }

    /* public constants */
    public static final double CREDITS_TO_GRADUATE = 32.0;
    /* Private instance variables */
    private String studentName;        /* The student's name */
    private int studentId;             /* The student's id */
    private double creditsEarned;      /* The number of credits earned */
    private boolean paidUp;            /* Whether student is paid up */
}
```

Defining length

- Use named constant to declare the length of an array.

```
private static final int N_JUDGES = 5;
double[] scores = new double[N_JUDGES];
```

- Or read the length of an array from the user.

Java: array operations

- Use named constant to declare the length of an array, or read the length of an array from the user

```
private static final int N_JUDGES  =  5;
double[ ]  scores  =  new double[N_JUDGES];
```

- Identifying an element by an integer number or an expression

```
array[index], array[(a+b)/2]
```

- Cycling through array elements

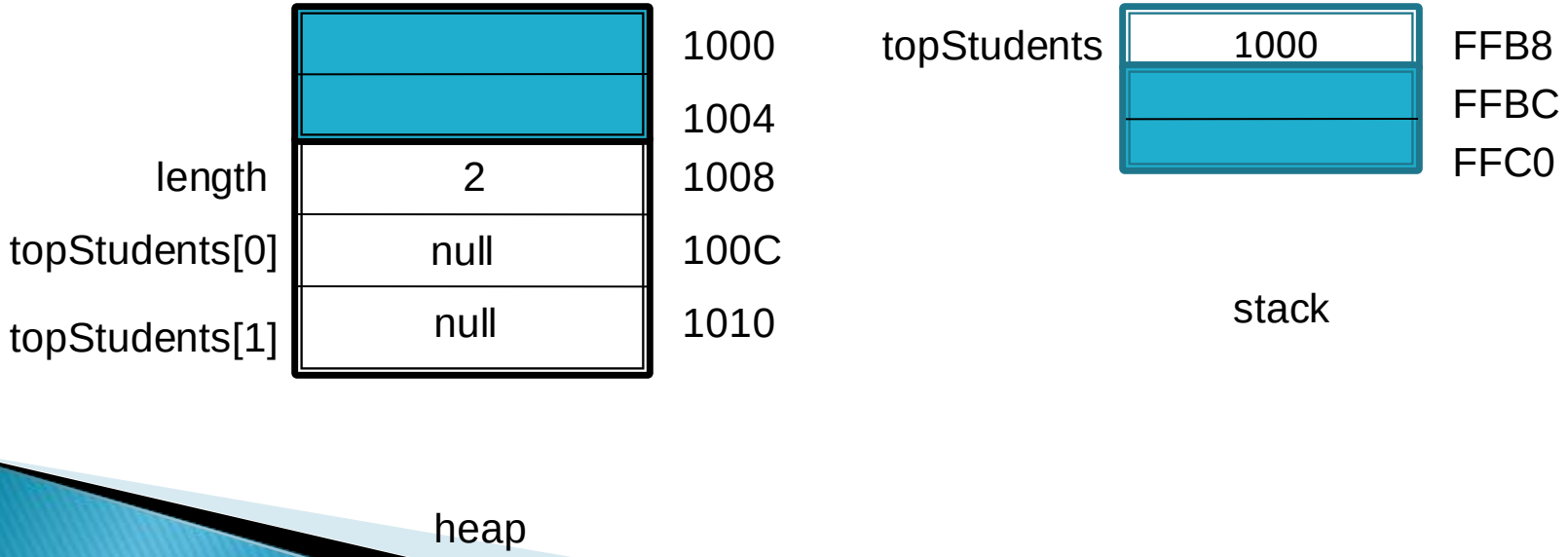
```
for (int i = 0; i < array.length; i++) {
      operations involving the ith element;
}
for (int value:array)
{
      operations involving values;
}
```

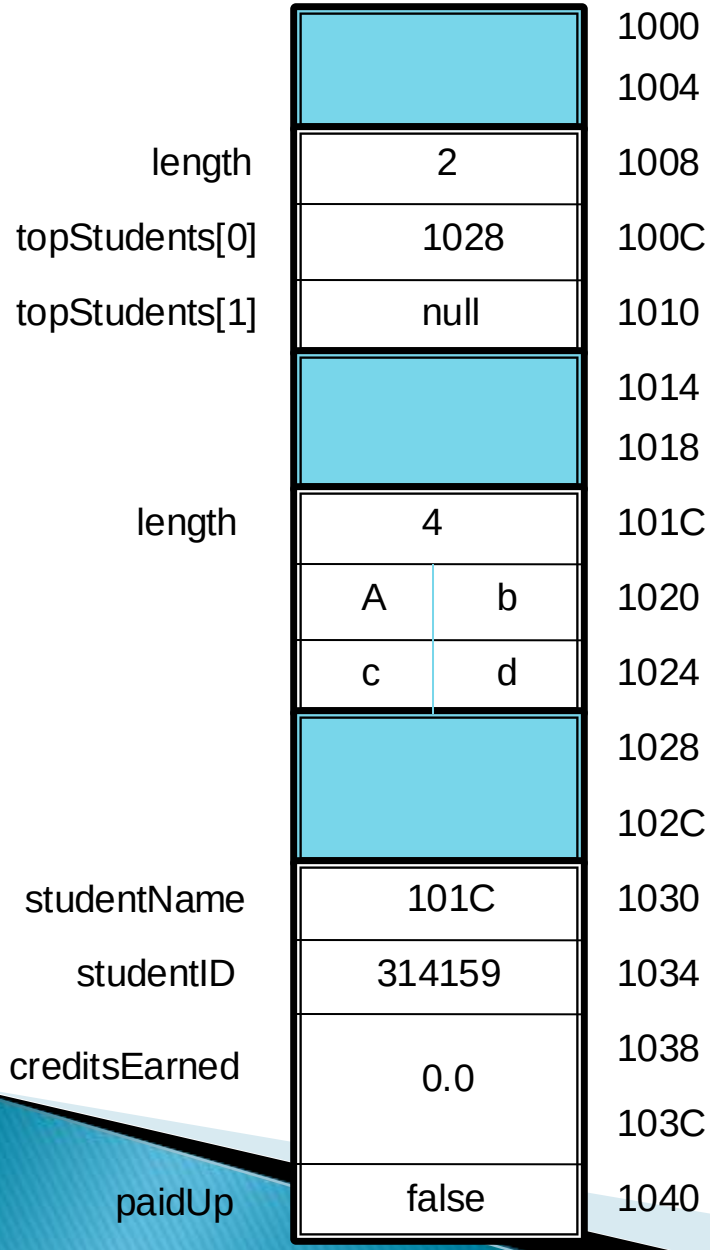
Human-readable index values

- Two typical starting numbers of indexing
  - 0: Java, Python, C/C++, …
  - 1: Matlab, Julia, R, …
- Nearly all Mathematics focused programming languages use 1-based indexing
- Starting index numbering at 0 can be unnatural.
  - Sometimes, it makes sense to work with index that begins with 1.
- Two standard ways:
  - Use Java's index number internally and then add one when presenting to the user.
  - Use index values beginning at 1 and ignore the first (0) element in each array.

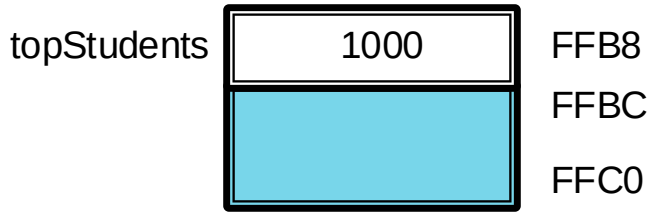
Internal representation of arrays

```
Student[ ]  topStudents  =  new Student[2];
topStudents[0] = new Student("Abcd", 314159);
```






```
Student[]  topStudents = new Student[2];
topStudents[0] = new Student("Abcd", 314159);
```



Passing arrays as parameters

- Recall: Passing objects (references) versus primitive type (values) as parameters.

- Java defines all arrays as objects, implying that the elements of an array are shared between the callee and the caller.

```
swapElements(array[i], array[n - i - 1])
```
(wrong)

```
swapElements(array, i, n - i - 1)
```

```
private void swapElements(int[] array, int p1, int p2) {
            int tmp = array[p1];
            array[p1] = array[p2];
            array[p2] = tmp;
}
```

- Every array in Java has a length field.

```
private void reverseArray(int[] array)  {
            for (int  i = 0; i < array.length / 2; i++) {
                    swapElements(array, i, array.length - i - 1);
            }
}
```

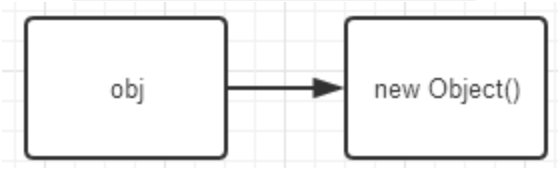
Pass-by-Value vs Pass-by-Reference

- What is Pass-by-Value?
  - The value of a function parameter is copied to another location of the memory
  - When accessing or modifying the variable within the function, it accesses only the copy, so there is no effect on the original value

- What is Pass-by-Reference?
  - The memory address is passed to that function, so the function gets access to the actual variable

Examples

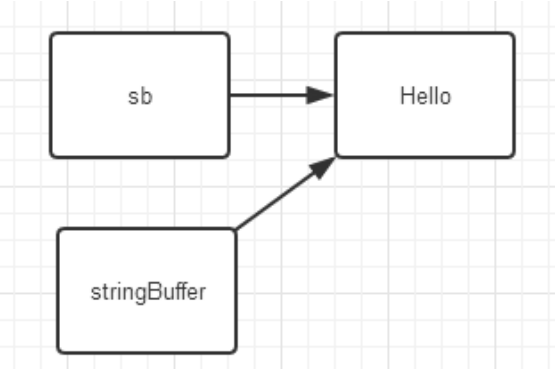
```
Object obj = new Object();
```




```
public class test {
    public static void main(String[] args) {
        int i = 1;
        System.out.println("before change, i = "+i);
        change(i);
        System.out.println("after change, i = "+i);
    }
    public static void change(int i){
        i = 5;
    }
}
```

```
before change, i = 1
after change, i = 1
```

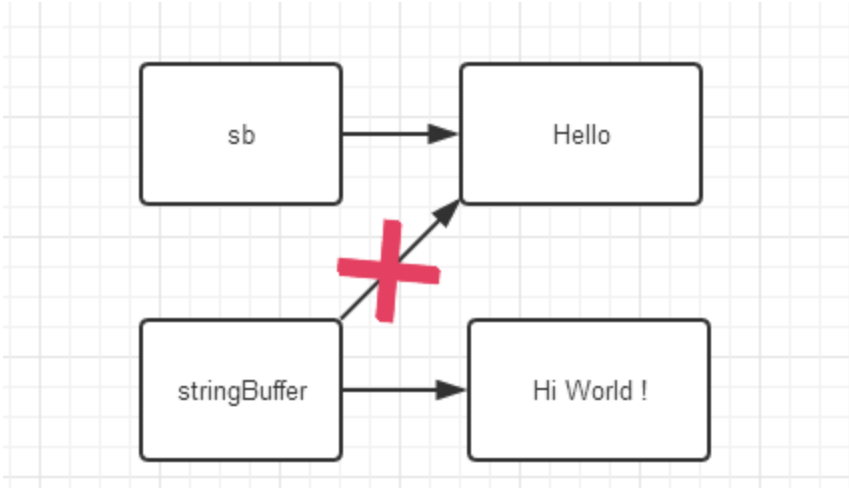
```
public class test {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello ");
        System.out.println("before change, sb is "+sb.toString());
        change(sb);
        System.out.println("after change, sb is "+sb.toString());
    }
    public static void change(StringBuffer stringBuffer){
        stringBuffer.append("world !");
    }
}
```




```
before change, sb is Hello
after change, sb is Hello world !
```

Examples

```
public class test {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello ");
        System.out.println("before change, sb is "+sb.toString());
        change(sb);
        System.out.println("after change, sb is "+sb.toString());
    }
    public static void change(StringBuffer stringBuffer){
        stringBuffer = new StringBuffer("Hi ");
        stringBuffer.append("world !");
    }
}
```




```
before change, sb is Hello
after change, sb is Hello
```

Examples

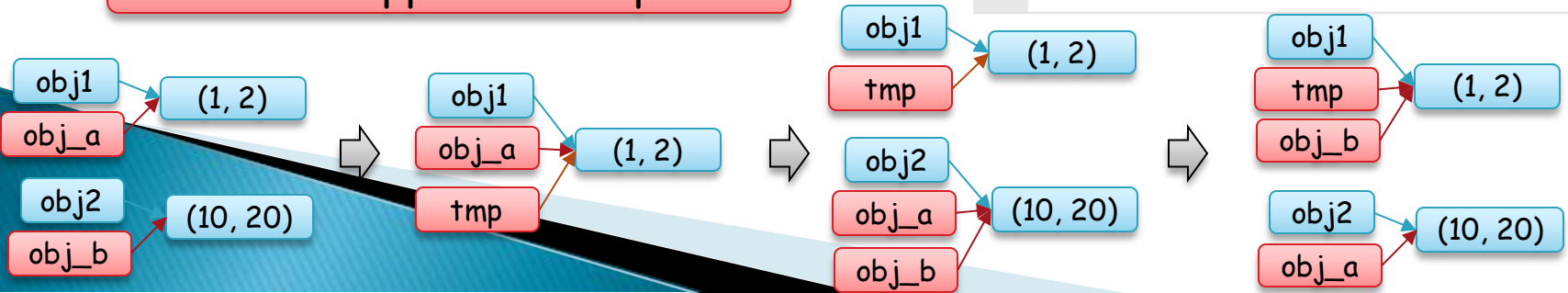
```
# obj_a: x=10 y=20
# obj_b: x=1 y=2

obj1: x=1 y=2
obj2: x=10 y=20
```

```
# obj_a: x=10 y=20
# obj_b: x=1 y=2

obj1: x=10 y=20
obj2: x=1 y=2
```

What happens in swap ?



Using arrays

- Example: Letter frequency table

Array: letterCounts[ ]

index: distance from ‘A’

index = Character.toUpperCase(ch) – ‘A’

letterCounts[0] is the count for ‘A’ or ‘a’

A convenient way of initializing an array:

```
int[] digits = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

private static final String[] US_CITIES_OVER_ONE_MILLION = {
  “New York”,
  “Los Angeles”,
  “Chicago”,
  “Huston”,
  “Philadelphia”,
  “Phoenix”,
  “San Diego”,
  “San Antonio”,
  “Dallas”,
}
```

Two-dimensional arrays

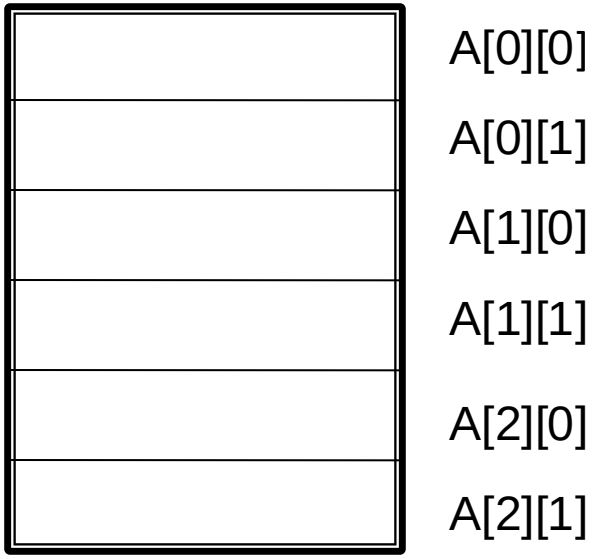
- Each element of an array is an array (of the same dimension)

```
int[][] A = new int[3][2];
```

- An array of three arrays of dimension two

```
A[0][0]   A[0][1]
A[1][0]   A[1][1]
A[2][0]   A[2][0]
```

Memory allocation (row orientation)



Initializing a two-dimensional array

```
static int A[3][2] = {
  {1, 4},
  {2, 5},
  {3, 6}
};
```

A 3-by-2 matrix

Remark. If a variable is static, it's value is shared between all usages (i.e. between all instances of the class).

The `ArrayList` Class

- The **`java.util`** package includes a class called **`ArrayList`** that provides the standard array behavior along with other useful operations.

- **`ArrayList`** is a Java class rather than a special form in the language. All operations on **`ArrayList`**s are indicated using method calls.
  - Create a new **`ArrayList`** by calling the **`ArrayList`** constructor.
  - Get the number of elements by calling the **`size`** method.
  - Use the **`get`** and **`set`** methods to select individual elements.

https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html

outline

- Basics in Computer Memory
- Arrays
- Implementation
- Examples

Typical operations on array

- Traversal: processing each element
- Search: finding an element by using the given index or key
- Insertion: adding new elements
- Deletion: removing elements
- Sorting: arranging elements with a type of order
- Merging: combining two arrays into a single array

These operations should also be considered for “linked list”

Insertion

- Insertion is NOT possible without loss of data if array is full
- We want to add element at a given location
- Let's create a new instance with length+1 for insertion
- Is this method safe?

```
public static int[] insertX(int n, int arr[],
                                    int x, int pos)
{
    int i;

    // create a new array of size n+1
    int newarr[] = new int[n + 1];

    // insert the elements from
    // the old array into the new array
    // insert all elements till pos
    // then insert x at pos
    // then insert rest of the elements
    for (i = 0; i < n + 1; i++) {
        if (i < pos - 1)
            newarr[i] = arr[i];
        else if (i == pos - 1)
            newarr[i] = x;
        else
            newarr[i] = arr[i - 1];
    }
    return newarr;
}
```

Deletion

- No data can be deleted from an empty array
- We want to delete an element at any index
- Let's create a new instance with length-1 for insertion

```
public static int[] removeTheElement(int[] arr, int index) {
    // If the array is empty or the index is not in array range
    // return the original array
    if (arr == null || index < 0 || index >= arr.length) {
        return arr;
    }

    // Create another array of size one less
    int[] anotherArray = new int[arr.length - 1];

    // Copy the elements except the index
    // from original array to the other array
    for (int i = 0, k = 0; i < arr.length; i++) {

        // if the index is the removal element index
        if (i == index) {
            continue;
        }

        // if the index is not the removal element index
        anotherArray[k++] = arr[i];
    }
    // return the resultant array
    return anotherArray;
}
```

Overview

- Basics in Computer Memory
- Arrays
- Implementation
- Examples
  - Comparison
  - Sorting
    - Selection sort, insertion sort, bubble sort
    - Quick sort

Testing Two Arrays for Equality

- To test two arrays for equality you need to define an `equals` method that returns true if and only the arrays have the same length and all corresponding values are equal
- This segment works for primitive types

```
public static boolean equals(int[] a,
                             int[] b)
   {
       boolean match = false;
       if (a.length == b.length)
       {
           match = true; //tentatively
           int i = 0;
           while (match && (i < a.length))
           {
               if (a[i] != b[i])
                   match = false;
               i++;
           }
       }
       return match;
    }
```

Methods that Return an Array

- the address of the array is passed
- The local array name within the method is just another name for the original array

```
public class returnArrayDemo
  {
      public static void main(String arg[])
      {
          char[] c;
          c = vowels();
          for(int i = 0; i < c.length; i++)
             System.out.println(c[i]);
       }
       public static char[] vowels()
       {
           char[] newArray = new char[5];
           newArray[0] = 'a';
           newArray[1] = 'e';
           newArray[2] = 'i';
           newArray[3] = 'o';
           newArray[4] = 'u';
           return newArray;
       }
  }
```

`c`, `newArray`, and the return type of `vowels` are all the same type: `char []`

```java
public class Main
{
    public static void main(String[] args)
    {
        // create an array
        int[] age = {12, 4, 5};

        // loop through the array
        // using for loop
        System.out.println("Using for-each Loop:");
        for(int a : age)
        {
            System.out.println(a);
        }
    }
}
```

Output

```
Using for-each Loop:
12
4
5
```

```java
public class Main
{
    public static void main(String[] args)
    {

        int[] numbers = {2, -9, 0, 5, 12, -25, 22, 9, 8, 12};
        int sum = 0;
        Double average;

        // access all elements using for each loop
        // add each element in sum
        for (int number : numbers)
        {
            sum += number;
        }

        // get the total number of elements
        int arrayLength = numbers.length;

        // calculate the average
        // convert the average from int to double
        average =  ((double)sum / (double)arrayLength);

        System.out.println("Sum = " + sum);
        System.out.println("Average = " + average);
    }
}
```

Output:

```
Sum = 36
Average = 3.6
```

```java
public class MultidimensionalArray {
    public static void main(String[] args)
    {

        int[][] a =
        {
            {1, -2, 3},
            {-4, -5, 6, 9},
            {7},
        };

        for (int i = 0; i < a.length; ++i)
        {
            for(int j = 0; j < a[i].length; ++j)
            {
                System.out.println(a[i][j]);
            }
        }
    }
}
```

Output:

```
1
-2
3
-4
-5
6
9
7
```

```java
public class MultidimensionalArray
{
    public static void main(String[] args)
    {

        // create a 2d array
        int[][] a =
        {
            {1, -2, 3},
            {-4, -5, 6, 9},
            {7},
        };

        // first for...each loop access the individual array
        // inside the 2d array
        for (int[] innerArray : a)
        {
            // second for...each loop access each element inside the row
            for(int data : innerArray)
            {
                System.out.println(data);
            }
        }
    }
}
```

Output:

```
1
-2
3
-4
-5
6
9
7
```

```java
public class ThreeArray
{
    public static void main(String[] args)
    {

        // create a 3d array
        int[][][] test =
        {
            {
                {1, -2, 3},
                {2, 3, 4}
            },
            {
                {-4, -5, 6, 9},
                {1},
                {2, 3}
            }
        };

        // for..each loop to iterate through elements of 3d array
        for (int[][] array2D : test)
        {
            for (int[] array1D : array2D)
            {
                for(int item : array1D)
                {
                    System.out.println(item);
                }
            }
        }
    }
}
```

Output:

```
1
-2
3
2
3
4
-4
-5
6
9
1
2
3
```

Examples: String array

- String array is an array holding a fixed number of strings or string values
  - One structure commonly used in Java
  - Even the arguments of the Java ‘main’ function is a String Array
    - public static void main(String args[ ]){…}

- String array is an array of String objects
  - Each string is an object!

```java
public class Main
{
    public static void main(String[] args)
    {

        String[] myarray;       //declaration of string array without size
        String[] strArray = new String[5];  //declaration with size

        //System.out.println(myarray[0]);   //variable myarray might not have been initialized
        //display elements of second array
        System.out.print(strArray[0] + " " + strArray[1] + " " + strArray[2] + " " +
                         strArray[3] + " " + strArray[4]);
    }
}
```

```
null null null null null
```

```java
import java.util.*;

public class Main
{
    public static void main(String[] args)
    {
        //original array
        String[] colorsArray = {"Red", "Green", "Blue" };
        System.out.println("Original Array: " + Arrays.toString(colorsArray));

        //length of original array
        int orig_length = colorsArray.length;
        //new element to be added to string array
        String newElement = "Orange";
        //define new array with length more than the original array
        String[] newArray = new String[ orig_length + 1 ];
        //add all elements of original array to new array
        for (int i = 0; i < colorsArray.length; i++)
        {
            newArray[i] = colorsArray [i];
        }
        //add new element to the end of new array
        newArray[newArray.length - 1] = newElement;
        //make new array as original array and print it
        colorsArray = newArray;
        System.out.println("Array after adding new item: " + Arrays.toString(colorsArray));
    }
}
```

```
Original Array: [Red, Green, Blue]

Array after adding new item: [Red, Green, Blue, Orange]
```

```java
import java.util.*;

class Main
{

    public static void main(String[] args)
    {
        String[] colors = {"red", "green", "blue", "white", "orange"};
        System.out.println("Original array: " + Arrays.toString(colors));
        Arrays.sort(colors);
        System.out.println("Sorted array: " + Arrays.toString(colors));
    }
}
```

```
Original array: [red, green, blue, white, orange]

Sorted array: [blue, green, orange, red, white]
```

```java
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        String[] strArray = { "Book", "Pencil", "Eraser", "Color", "Pen" };
        boolean found = false;
        int index = 0;
        String searchStr = "Pen";
        for (int i = 0; i < strArray.length; i++)
        {
            if(searchStr.equals(strArray[i]))
            {
                index = i;
                found = true;
                break;
            }
        }
        if(found)
            System.out.println(searchStr + " found at the index " + index);
        else
            System.out.println(searchStr + " not found in the array");

    }
}
```

```
Pen found at the index 4
```

```java
import java.util.*;

public class Main
{
    public static void main( String[] args )
    {
        //string arrya declaration
        String [] str_Array = {"10", "20", "30", "40", "50"};
        //print the string array
        System.out.println("Original String Array:");
        for(String val : str_Array)
            System.out.print(val + " ");

        System.out.println("\nThe integer array obtained from string array:");
        //declare an int array
        int [] int_Array = new int [str_Array.length];
        //assign string array values to int array
        for(int i = 0; i < str_Array.length; i++)
        {
            int_Array[i] = Integer.parseInt(str_Array[i]);
        }
        //display the int array
        System.out.println(Arrays.toString(int_Array));
    }
}
```

```
Original String Array:

10 20 30 40 50

The integer array obtained from string array:

[10, 20, 30, 40, 50]
```

Sorting an Array

- Sorting a list of elements is another very common problem (along with searching a list)
  - sort numbers in ascending order
  - sort numbers in descending order
  - sort strings in alphabetic order
  - etc.

- There are many ways to sort a list, just as there are many ways to search a list

- Selection sort, insertion sort, bubble sort
- QuickSort

Selection Sort

To sort an array on integers in ascending order:

1. Find the smallest number and record its index
2. swap (interchange) the smallest number with the first element of the array
   - the sorted part of the array is now the first element
   - the unsorted part of the array is the remaining elements
3. repeat Steps 2 and 3 until all elements have been placed
   - each iteration increases the length of the sorted part by one

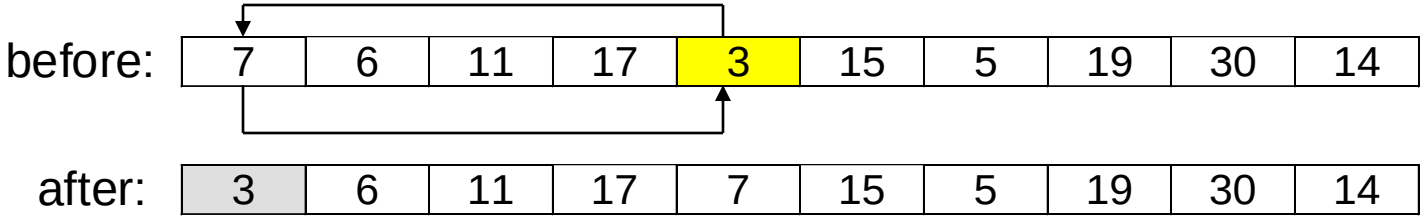
Selection Sort Example

Key:
- smallest remaining value
- sorted elements

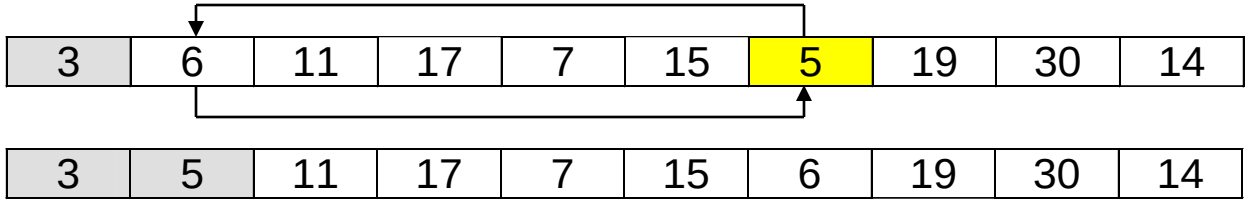
Problem: sort this 10-element array of integers in ascending order:

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
7	6	11	17	3	15	5	19	30	14

1st iteration: smallest value is 3, its index is 4, swap a[0] with a[4]



2nd iteration: smallest value in remaining list is 5, its index is 6, swap a[1] with a[6]



How many iterations are needed?

Example: Selection Sort

- Notice the precondition: every array element has a value
- may have duplicate values
- broken down into smaller tasks
  - "find the index of the smallest value"
  - "interchange two elements"
  - `private` because they are helper methods (users are not expected to call them directly)

Selection Sort Code

```
/*********************************
*Precondition:
*Every indexed variable of the array a has a value.
*Action: Sorts the array a so that
*a[0] <= a[1] <= ... <= a[a.length - 1].
*****************************************/
public static void sort(int[] a)
{
    int index, indexOfNextSmallest;
    for (index = 0; index < a.length - 1; index++)
    {//Place the correct value in a[index]:
        indexOfNextSmallest = indexOfSmallest(index, a);
        interchange(index,indexOfNextSmallest, a);
        //a[0] <= a[1] <=...<= a[index] and these are
        //the smallest of the original array elements.
        //The remaining positions contain the rest of
        //the original array elements.
    }
}
```

Question

- Write Java code for methods **`indexOfSmallest`** and **`interchange`**
- What is the big-O complexity? N refers to the length of the array.

Insertion Sort

- Basic Idea:
  - Keeping expanding the sorted portion by one
  - Insert the next element into the right position in the sorted portion
- Algorithm:
  1. Start with one element [is it sorted?] – sorted portion
  2. While the sorted portion is not the entire array
     1. Find the right position in the sorted portion for the next element
     2. Insert the element
     3. If necessary, move the other elements down
     4. Expand the sorted portion by one
- $O(N^2)$

Insertion Sort: An example

- First iteration
  - Before: **[5]**, 3, 4, 9, 2
  - After: **[**3, **5]**, 4, 9, 2
- Second iteration
  - Before: **[3, 5]**, 4, 9, 2
  - After: **[3,** 4, **5]**, 9, 2
- Third iteration
  - Before: **[3, 4, 5]**, 9, 2
  - After: **[3, 4, 5,** 9], 2
- Fourth iteration
  - Before: **[3, 4, 5, 9]**, 2
  - After: **[**2, **3, 4, 5, 9]**

Question

- Recall `insert` function in previous lecture
- Can you implement `insertSort` by modifying that function (without creating new instance) ?

```
public static int[] insertX(int n, int arr[],
                                    int x, int pos)
{
    int i;

    // create a new array of size n+1
    int newarr[] = new int[n + 1];

    // insert the elements from
    // the old array into the new array
    // insert all elements till pos
    // then insert x at pos
    // then insert rest of the elements
    for (i = 0; i < n - 1; i++) {
        if (i < pos - 1)
            newarr[i] = arr[i];
        else if (i == pos - 1)
            newarr[i] = x;
        else
            newarr[i] = arr[i - 1];
    }
    return newarr;
}
```

Bubble Sort

- Basic Idea:
  - Expand the sorted portion one by one
  - “Sink” the largest element to the bottom after comparing adjacent elements
  - The smaller items “bubble” up
- Algorithm:
  - While the unsorted portion has more than one element
    - Compare adjacent elements
    - Swap elements if out of order
    - Largest element at the bottom, reduce the unsorted portion by one
- $O(N^2)$

Bubble Sort: An example

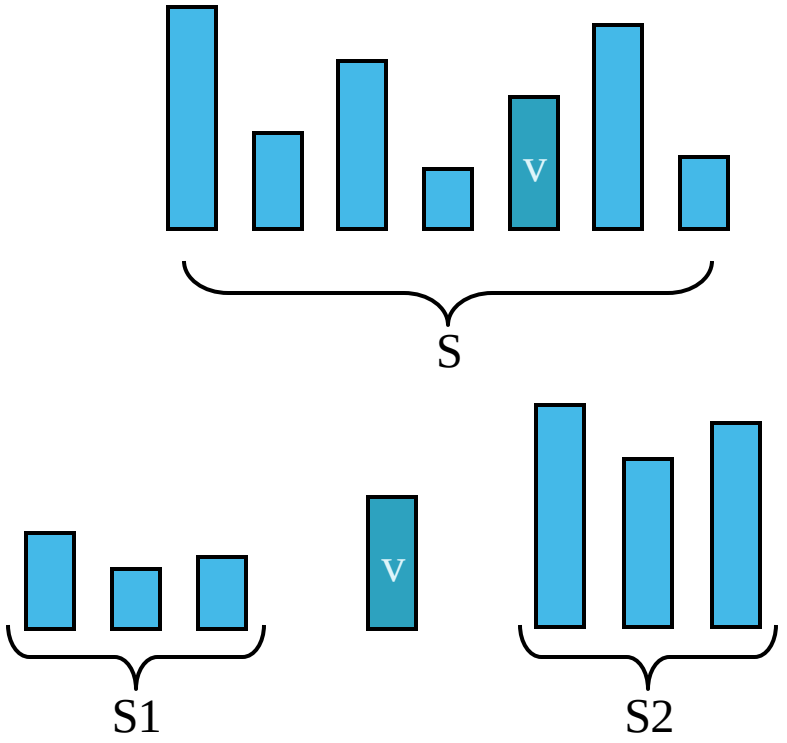
- First Iteration:
  - [5, 3], 4, 9, 2 → [3, 5], 4, 9, 2
  - 3, [5, 4], 9, 2 → 3, [4, 5], 9, 2
  - 3, 4, [5, 9], 2 → 3, 4, [5, 9], 2
  - 3, 4, 5, [9, 2] → 3, 4, 5, [2, 9]
- Second Iteration:
  - [3, 4], 5, 2, **9** → [3, 4], 5, 2, **9**
  - 3, [4, 5], 2, **9** → 3, [4, 5], 2, **9**
  - 3, 4, [5, 2], **9** → 3, 4, [2, 5], **9**
- Third Iteration:
  - [3, 4], 2, **5, 9** → [3, 4], 2, **5, 9**
  - 3, [4, 2], **5, 9** → 3, [2, 4], **5, 9**
- Fourth Iteration:
  - [3, 2], **4, 5, 9** → [2, 3], **4, 5, 9**

QuickSort

- Fastest known sorting algorithm in practice
- Average case: O(N log N) (we don't prove it)
- Worst case: $O(N^2)$
  - But, the worst case seldom happens.
- Another divide-and-conquer recursive algorithm, like mergesort
- We will discuss more about recursion later in this course

Quicksort

- Divide step:
  - Pick any element (*pivot*) v in S
  - Partition S – {v} into two disjoint groups
    S1 = {x ∈ S – {v} | x <= v}
    S2 = {x ∈ S – {v} | x ≥ v}
- Conquer step: recursively sort S1 and S2
- Combine step: the sorted S1 (by the time returned from recursion), followed by v, followed by the sorted S2 (i.e., nothing extra needs to be done)



To simplify, we may assume that we don't have repetitive elements, So to ignore the 'equality' case!

Pseudo-code (Wiki)

```
algorithm quicksort(A, lo, hi) is
    if lo < hi then
        p := partition(A, lo, hi)
        quicksort(A, lo, p - 1)
        quicksort(A, p + 1, hi)

algorithm partition(A, lo, hi) is
    pivot := A[hi]
    i := lo
    for j := lo to hi - 1 do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```

Two key steps

- How to pick a pivot?
- How to partition?



Pick a pivot

- Use the first element as pivot
  - if the input is random, ok
  - if the input is presorted (or in reverse order)
    - all the elements go into S2 (or S1)
    - this happens consistently throughout the recursive calls
- Choose the pivot randomly
  - generally safe
  - random number generation can be expensive

In-place Partition

- If use additional array (not in-place)
  - Straightforward to code
  - Inefficient!

- Many ways to implement
- Even the slightest deviations may cause surprisingly bad results.
  - Not stable as it does not preserve the ordering of the identical keys.
  - Hard to write correctly ☹

An easy version of in-place partition to understand,
but not the original form

```
int partition(arr, left, right, pivotIndex) {
   pivotValue = arr[pivotIndex];
   swap(arr[pivotIndex], arr[right]); // Move pivot to end

   // move all smaller (than pivotValue) to the begining
   storeIndex = left;
   for (i from left to right) {
        if a[i] < pivotValue
               swap(a[storeIndex], arr[i]);
               storeIndex = storeIndex + 1 ;
   }

   swap(arr[right], arr[storeIndex]); // Move pivot to its final place
   return storeIndex;
}
```

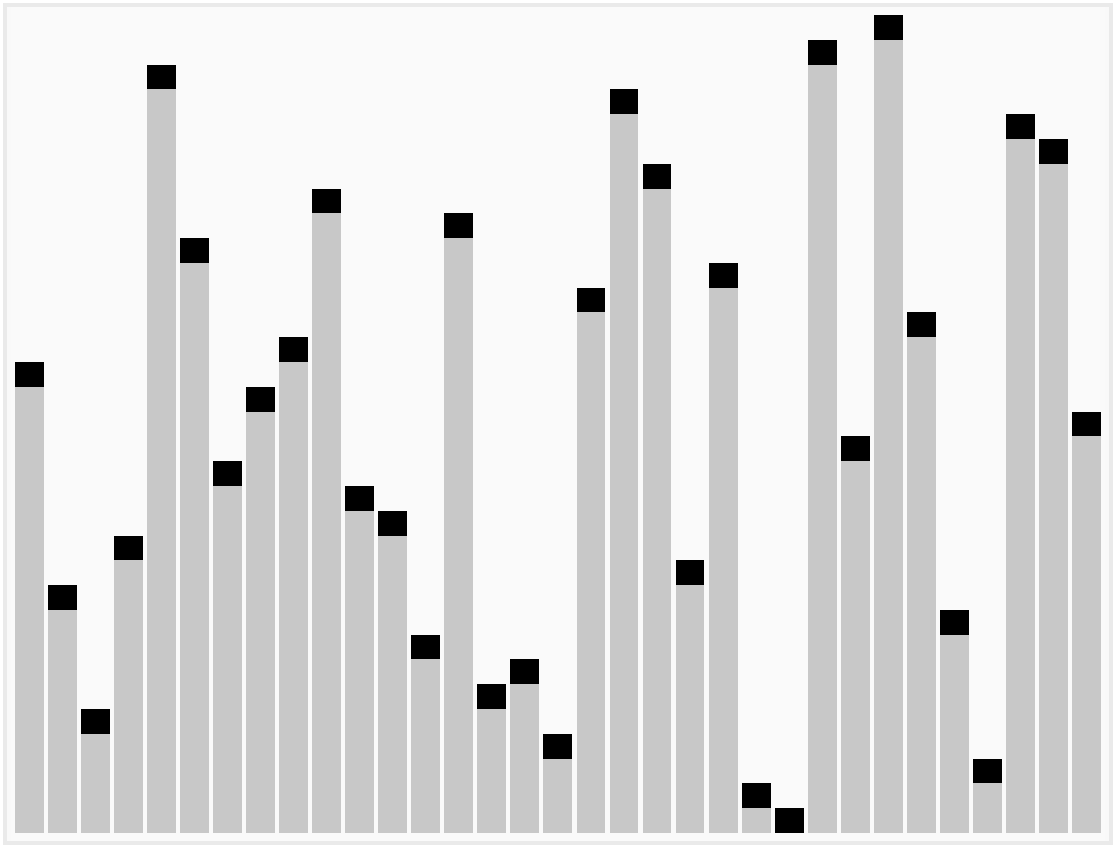
Look at Wikipedia

```
quicksort(a,left,right) {

   if (right>left) {
   pivotIndex = left;
   select a pivot value a[pivotIndex];

   pivotNewIndex=partition(a,left,right,pivotIndex);

   quicksort(a,left,pivotNewIndex-1);
   quicksort(a,pivotNewIndex+1,right);
   }
}
```



https://en.wikipedia.org/wiki/Quicksort

Implementation (put the pivot on the leftmost instead of rightmost)

```
void quickSort(int array[], int start, int end)
{
     int i = start; // index of left-to-right scan
     int k = end; // index of right-to-left scan

     if (end - start >= 1) // check that there are at least two elements to sort
     {
            int pivot = array[start]; // set the pivot as the first element in the partition
            while (k > i) // while the scan indices from left and right have not met,
            {
                        while (array[i] <= pivot && i <= end && k > i)  // from the left, look for the first
                                    i++;                                                // element greater than the pivot
                        while (array[k] > pivot && k >= start && k >= i) // from the right, look for the first
                                    k--;                                                // element not greater than the pivot
                        if (k > i)                                          // if the left seekindex is still smaller than
                                    swap(array, i, k);                                  // the right index,
                                                                                        // swap the corresponding elements
            }
            swap(array, start, k);                                          // after the indices have crossed,
                                                                                        // swap the last element in
                                                                                        // the left partition with the pivot
            quickSort(array, start, k - 1);                                 // quicksort the left partition
            quickSort(array, k + 1, end);                                   // quicksort the right partition
     }
     else // if there is only one element in the partition, do not do any sorting
     {
      return; // the array is sorted, so exit
     }
}
```

Adapted from http://www.mycsresource.net/articles/programming/sorting_algos/quicksort/

Sorting Algorithm Tradeoffs

- Easy to understand algorithms
  - not very efficient
  - less likely to have mistakes
  - require less time to code, test, and debug
  - Selection, Insertion, Bubble Sorting algorithms
  - Bubble Sort is the easiest to implement
- Complicated but more efficient
  - useful when performance is a major issue
  - programming project for Chapter 11 describes a more efficient sorting algorithm

"Getting the wrong result is always inefficient."



Jagged Arrays

- Jagged arrays have rows of unequal length
  - each row has a different number of columns, or entries

- Jagged arrays are allowed in Java

- Example: create a 2-D `int` array named `b` with 5 elements in the first row, 7 in the second row, and 4 in the third row:

```
int[][] b = new int[3][];
b[0] = new int[5];
b[1] = new int[7];
b[2] = new int[4];
```

Initializing jagged arrays

```
int arr_name[][] = new int[][] {
      new int[] {10, 20, 30 ,40},
      new int[] {50, 60, 70, 80, 90, 100},
      new int[] {110, 120}
};

int[][] arr_name = {
      {10, 20, 30 ,40},
      {50, 60, 70, 80, 90, 100},
      {110, 120}
};
```